

MODELO DE DISEÑO

Proyecto #2 — Dulces & Dados Board Game Café

Grupo 7

ISIS-1226 · Diseño y Programación Orientada a Objetos

1. Introducción

El presente documento describe el diseño detallado del sistema desarrollado para el Board Game Café "Dulces & Dados", integrando los avances del Proyecto 1 y las extensiones incorporadas en el Proyecto 2. El objetivo es ofrecer una visión completa y continua del sistema de software, desde su concepción inicial hasta su estado actual, incluyendo la gestión de operaciones del café, el módulo de torneos, las interfaces de consola por rol, las pruebas automatizadas y la persistencia de datos.

El sistema permite la interacción de tres tipos principales de usuarios: clientes, empleados y administrador. En el Proyecto 1 se modelaron y estructuraron las funcionalidades base del café: reservas de mesas, préstamos de juegos, ventas, gestión de inventario y administración de turnos. En el Proyecto 2 se amplió el sistema con un módulo completo de torneos, interfaces por consola, persistencia automática en archivos y pruebas unitarias e integración.

Este documento presenta los modelos utilizados para representar el sistema, así como las decisiones de diseño tomadas durante ambas etapas de desarrollo, con el fin de garantizar coherencia entre el análisis, el diseño y la implementación.

2. Descripción General del Sistema

El sistema permite la interacción de tres tipos de usuarios con roles bien diferenciados: clientes, empleados y administrador. A través del sistema se gestionan los siguientes procesos:

- Reserva de mesas por parte de los clientes.
- Préstamo de juegos de mesa, con validaciones de disponibilidad, edad y restricciones por mesa.
- Compra de productos de cafetería y juegos de la tienda.
- Gestión de inventario de préstamo y venta por parte del administrador.
- Administración de turnos de los empleados y solicitudes de cambio.
- Creación y participación en torneos amistosos y competitivos.
- Persistencia automática del estado del sistema en archivos de texto plano.

El sistema garantiza el cumplimiento de las reglas del negocio, tales como restricciones de edad, disponibilidad de juegos, control de capacidad del establecimiento, condiciones de consumo, reglas de inscripción a torneos y otorgamiento de premios.

3. Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas con separación clara de responsabilidades, lo que facilita su mantenimiento, extensión y prueba de forma independiente:

3.1 Capa de Presentación

Representada por las clases `MainAdministrador`, `MainCliente` y `MainEmpleado`, cada una con su propio método `main` que define el flujo de interacción según el rol. La clase auxiliar `ValidadorEntradaConsola` centraliza la lógica de validación de entradas del usuario, evitando duplicación entre los tres flujos. Esta capa solo se comunica con la fachada `SistemasDulcesDados`, sin acceder directamente a las clases del dominio.

3.2 Capa de Lógica de Negocio

Representada por `SistemasDulcesDados` —que actúa como fachada principal— y por todas las clases del dominio: jerarquía de usuarios, torneos, café, juegos, préstamos, ventas, turnos y sugerencias. Esta capa concentra todas las reglas del negocio y es completamente independiente de la capa de presentación y de persistencia.

3.3 Capa de Persistencia

Representada por la clase `PersistenciaSistema`, encargada de la lectura y escritura de datos en archivos de texto plano. Implementa un mecanismo de caché interno que evita lecturas redundantes durante una sesión. Los archivos se organizan por entidad (usuarios, café, juegos, turnos, torneos, bonos, premios, entre otros) y se actualizan automáticamente al finalizar cada sesión.

4. Modelo de Clases — Proyecto 1: Base del Sistema

En esta sección se describen las clases diseñadas e implementadas durante el Proyecto 1, que constituyen el núcleo del sistema de gestión del café. Estas clases permanecen vigentes en el Proyecto 2, con las extensiones señaladas en la sección siguiente.

4.1 Jerarquía de Usuarios

La jerarquía de usuarios refleja los tres roles del sistema. La clase abstracta `Usuario` define los atributos y comportamientos comunes a todos los actores.

Usuario (abstracta)	
Descripción	Clase base que representa cualquier persona con acceso al sistema. Define credenciales e información de identificación compartida por todos los roles.
Atributos	<code>documentoIdentidad</code> , <code>nombre</code> , <code>correoElectronico</code> , <code>login</code> , <code>password</code>
Métodos clave	<code>iniciarSesion(login, password)</code> , <code>cerrarSesion()</code> , <code>validarPassword(password)</code>

Cliente	
Descripción	Usuario consumidor del café. Puede reservar mesas, solicitar préstamos de juegos, realizar compras, acumular puntos de fidelidad y guardar juegos favoritos. En el Proyecto 2 se le agregan los atributos <code>bonoDescuentoGanado</code> y <code>premioMonetarioPendiente</code> para el módulo de torneos.

Atributos	puntosFidelidad, juegosFavoritos, bonoDescuentoGanado (*), premioMonetarioPendiente (*) (*) Incorporados en Proyecto 2
Métodos clave	crearReserva(), solicitarPrestamo(), devolverPrestamo(), comprarProductos(), aplicarCodigoDescuento(), usarPuntosFidelidad(), acumularPuntos(), agregarJuegoFavorito(), recibirBonoDescuento(*), aplicarBonoAVenta(*)

Empleado (abstracta)

Descripción	Usuario que trabaja en el café. Distingue empleados de tipo Mesero y Cocinero. Gestiona turnos, solicitudes de cambio, compras internas y sugerencias. En el Proyecto 2 se le incorpora bonoDescuentoGanado para poder recibir premios en torneos amistosos.
Atributos	codigoEmpleado, enTurno, turnos, juegosFavoritos, bonoDescuentoGanado (*) (*) Incorporado en Proyecto 2
Métodos clave	consultarTurnos(), solicitarCambioTurno(), solicitarPrestamoEmpleado(), comprarProductos(), crearSugerenciaPlatillo(), agregarJuegoFavorito(), recibirBonoDescuento(*)

Mesero

Descripción	Empleado encargado de la atención en sala y operaciones en caja. Puede estar capacitado para explicar juegos difíciles; en caso contrario, el sistema genera una advertencia al cliente.
Atributos	(Asociación con JuegoMesa para juegos conocidos)
Métodos clave	registrarJuegoConocido(), eliminarJuegoConocido(), puedeExplicarJuego(), registrarVenta()

Cocinero

Descripción	Empleado encargado de preparar productos del menú. No puede asumir el rol de mesero. Puede sugerir nuevos platillos al administrador.
Atributos	(Hereda de Empleado)
Métodos clave	crearSugerenciaPlatillo() (heredado)

Administrador

Descripción	Usuario responsable de la gestión global: inventarios, turnos, aprobaciones y reportes. No puede pedir juegos prestados ni ordenar platillos. En el Proyecto 2 se le otorgan métodos para crear y gestionar torneos.
Atributos	(Hereda de Usuario)
Métodos clave	consultarInventarioPrestamo(), moverJuegoVentaAPrestamo(), reabastecerJuegoVenta(), repararJuego(), aprobarSugerencia(), asignarTurno(), generarInformeVentas(), crearTorneoAmistoso(*), crearTorneoCompetitivo(*), cancelarTorneo(*) (*) Incorporados en Proyecto 2

4.2 Operación del Café

Cafe	
Descripción	Entidad central del establecimiento. Administra capacidad máxima, mesas, empleados, catálogo de juegos, inventario de venta, menú, ventas, préstamos y — a partir del Proyecto 2— la lista de torneos.
Atributos	capacidadMaximaClientes, mesas, empleados, catalogoJuegos, inventarioVenta, menu, ventas, prestamos, torneos (*) (*) Incorporado en Proyecto 2
Métodos clave	verificarCapacidadDisponible(), buscarMesaDisponible(), agregarMesa(), registrarVenta(), registrarPrestamo(), buscarMeseroCapacitado(), generarInformeVentas(), agregarTorneo(*), consultarTorneosDisponibles(*), validarCreacionTorneo(*)

Mesa	
Descripción	Mesa física del café. Sirve para sentar clientes, validar restricciones de consumo y vincular préstamos. Controla presencia de menores, bebidas calientes y juegos de acción.
Atributos	numeroMesa, reservaActiva
Métodos clave	asignarReserva(), cerrarReservaActiva(), tieneMenoresEdad(), tieneNinosMenores5(), tieneBebidaCalienteActiva(), tieneJuegoAccionActivo(), admiteJuego(), puedeRecibirBebida()

ReservaMesa	
Descripción	Representa la asignación de una mesa a un grupo de clientes. Registra número de personas, presencia de niños y menores de edad, y controla el estado de la reserva.
Atributos	fechaHora, numeroPersonas, hayNinosMenores5, hayMenoresEdad, estadoReserva
Métodos clave	aceptar(), rechazar(), activar(), cerrar(), estaActiva(), validarCapacidad()

4.3 Juegos y Préstamos

JuegoMesa	
Descripción	Juego de mesa del catálogo de préstamo. Tiene nombre, año, empresa, rango de jugadores, edad mínima, categoría y si es difícil. Administra sus propias copias físicas.
Atributos	nombre, anioPublicacion, empresaMatriz, minJugadores, maxJugadores, edadMinima, dificil, categoria, copias
Métodos clave	esAptoParaCantidadJugadores(), esAptoParaEdad(), esDificil(), esCategoriaAccion(), tieneCopiasDisponibles(), obtenerCopiasDisponibles(), agregarCopia()

CopiaJuegoPrestamo	
Descripción	Copia física de un juego destinada a préstamo. Tiene estado (NUEVO, BUENO, FALTA_PIEZA, DESAPARECIDO) y disponibilidad. El administrador puede reparar o marcar copias como desaparecidas.
Atributos	estado, disponible
Métodos clave	estaDisponible(), prestar(), devolver(), cambiarEstado(), marcarDesaparecido(), marcarFaltaPieza()

Prestamo	
Descripción	Acto de prestar uno o dos juegos. Valida cantidad máxima, disponibilidad de copias, mesa del cliente, restricciones de edad y manejo de juegos difíciles.
Atributos	fechaInicio, fechaFin, advertenciaSinMesero
Métodos clave	agregarDetalle(), validarMaximoJuegos(), validarMesaRequerida(), validarRestriccionesMesa(), validarDisponibilidadCopias(), validarJuegosDificiles(), finalizarPrestamo(), estaActivo()

DetallePrestamo	
Descripción	Elemento individual de un préstamo, referencia a una copia física específica. Permite mantener historial de asignación y devolución por copia.
Atributos	fechaAsignacion, fechaDevolucion
Métodos clave	registrarAsignacion(), registrarDevolucion(), estaDevuelto()

4.4 Ventas y Productos

ProductoVendible (abstracta)	
Descripción	Abstracción común para cualquier ítem que pueda venderse: juego de tienda o producto del menú.
Atributos	nombre, precio, disponible
Métodos clave	estaDisponible(), obtenerPrecio(), calcularSubtotal(cantidad)

JuegoVenta	
Descripción	Juego disponible para compra en tienda. Su inventario es independiente del inventario de préstamo.
Atributos	stockDisponible
Métodos clave	hayStock(cantidad), reducirStock(cantidad), aumentarStock(cantidad)

ProductoMenu (abstracta)	
--------------------------	--

Descripción	Producto ofrecido en el menú del café. Agrupa bebidas y productos de pastelería.
Atributos	(Hereda de ProductoVendible)
Métodos clave	(Métodos heredados)

Bebida	
Descripción	Producto del menú que puede ser alcohólico y/o caliente. El sistema impide bebidas alcohólicas en mesas con menores de edad y bebidas calientes en mesas con juegos de acción activos.
Atributos	esAlcoholica, esCaliente
Métodos clave	esAlcoholica(), esCaliente(), puedeDespacharseAMesa(mesa)

Pastelería	
Descripción	Producto del menú que registra posibles alérgenos para advertir al cliente.
Atributos	(Asociación con Alergeno)
Métodos clave	contieneAlergeno(), obtenerAlergenos(), generarAdvertenciaAlergenos()

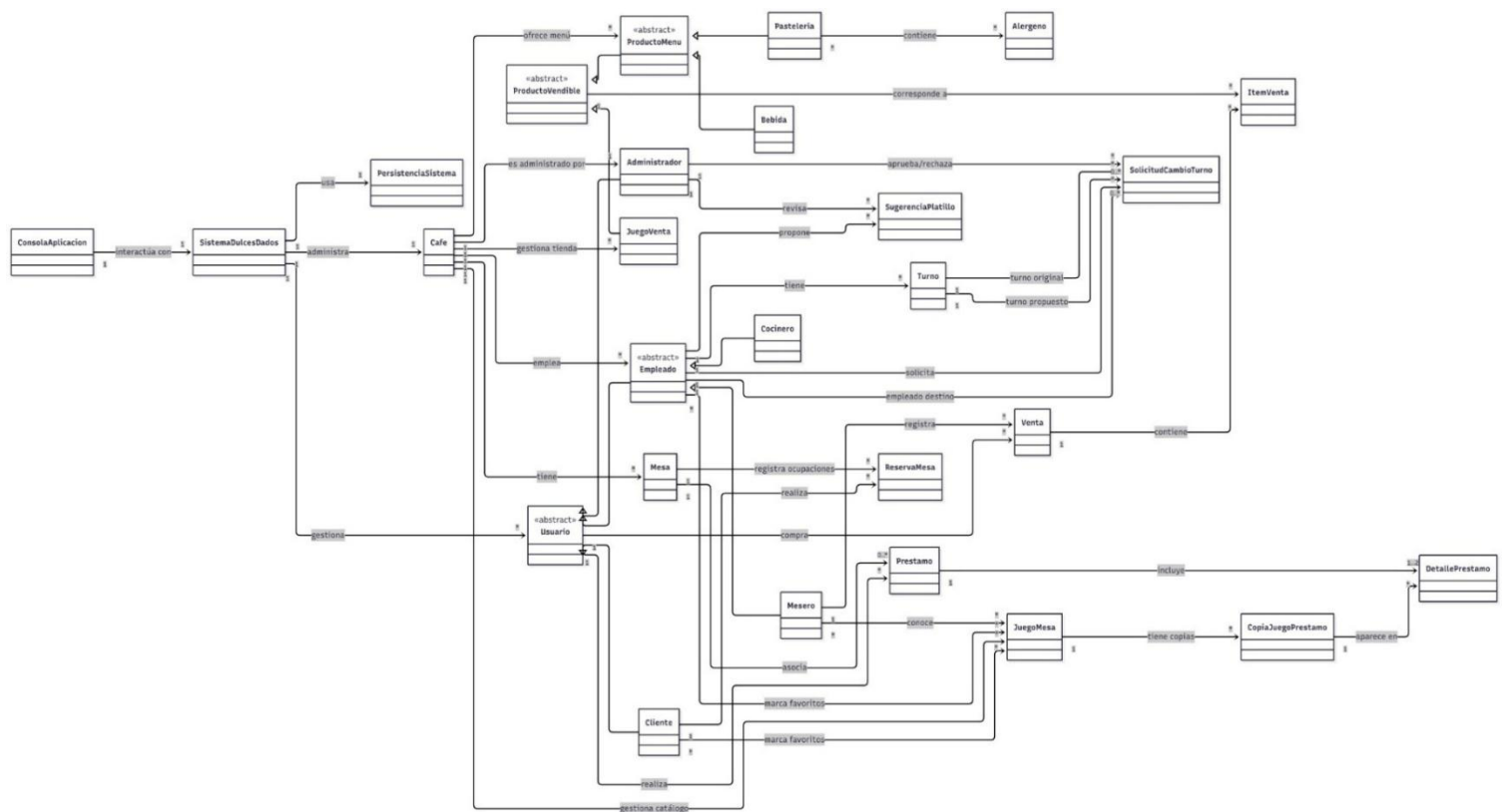
Venta	
Descripción	Transacción registrada en el sistema. Puede ser de cafetería o de tienda. Gestiona subtotal, descuentos, impuestos, propina, puntos de fidelidad y discriminación por rubros.
Atributos	fechaHora, tipoVenta, descuentoAplicado, propina
Métodos clave	agregarItem(), calcularSubtotal(), calcularDescuento(), calcularImpuestos(), calcularTotal(), aplicarDescuentoPorcentaje(), generarPuntosFidelidad()

ItemVenta	
Descripción	Elemento individual dentro de una venta. Registra producto, cantidad y precio unitario.
Atributos	cantidad, precioUnitario
Métodos clave	calcularSubtotal(), correspondeAProductoMenu(), correspondeAJuegoVenta()

4.5 Gestión Administrativa

SugerenciaPlatillo	
Descripción	Propuesta de un empleado para agregar un nuevo platillo al menú. Puede ser aprobada o rechazada por el administrador.
Atributos	nombrePropuesto, categoriaPropuesta, fechaHora, estado
Métodos clave	aprobar(), rechazar(), estaPendiente()

SolicitudCambioTurno	
Descripción	Solicitud de cambio o intercambio de turno entre empleados. Debe respetar el mínimo operativo del café (1 cocinero y 2 meseros simultáneos).
Atributos	tipoSolicitud, fechaHora, estado
Métodos clave	aprobar(), rechazar(), esIntercambio(), esCambioSimple(), validarMinimosOperacion(café)



5. Modelo de Clases — Proyecto 2: Módulo de Torneos

El Proyecto 2 incorporó al sistema un módulo completo de torneos. Se agregaron tres nuevas clases al modelo de dominio y se extendieron las clases Cliente, Empleado, Administrador y Cafe. A continuación se describe cada nueva clase.

5.1 Nuevas Clases del Módulo de Torneos

Torneo (abstracta)	
Descripción	Clase base del módulo de torneos. Concentra la lógica común de gestión de cupos, inscripciones, desinscripciones y validaciones de elegibilidad. El 20% de los cupos totales (redondeado hacia arriba) se reserva automáticamente para jugadores fanáticos del juego del torneo.
Atributos	nombre, dia (DiaSemana), juego (JuegoMesa), cuposTotales, cuposFanaticos, cuposFanaticosUsados, cuposRegularesUsados, inscripciones (List<InscripcionTorneo>)
Métodos clave	inscribir(usuario, numeroCupos), desinscribir(usuario), puedeInscribirse(), esFanatico(usuario), empleadoTieneTurnoEseDia(), estaLleno(), getCuposFanaticosDisponibles(), getCuposRegularesDisponibles(), obtenerDescripcionPremio() (abstracto), otorgarPremio(ganador) (abstracto)

TorneoAmistoso	
Descripción	Torneo sin tarifa de entrada. El ganador recibe un bono de descuento no acumulable para su próxima compra, aplicable tanto a clientes como a empleados.
Atributos	bonoDescuento (porcentaje, ej: 0.20 = 20%)
Métodos clave	obtenerDescripcionPremio(), otorgarPremio(ganador) → llama a recibirBonoDescuento() en el ganador

TorneoCompetitivo	
Descripción	Torneo con tarifa de entrada por participante (para clientes). El premio monetario se calcula como la suma de tarifas pagadas únicamente por clientes inscritos. Los empleados participan gratis y no reciben el premio monetario.
Atributos	tarifaEntrada (monto por participante cliente)
Métodos clave	calcularPremioMonetario() → suma cupos de clientes × tarifa, calcularTarifaParaUsuario(usuario) → 0 si es empleado, otorgarPremio(ganador) → solo clientes reciben el premio

InscripcionTorneo	
Descripción	Registra la inscripción de un usuario a un torneo. Guarda el usuario, cuántos cupos tomó y si utilizó el pool de cupos de fanático o el pool regular. Este flag es esencial para devolver correctamente los cupos al desinscribirse.
Atributos	usuario (Usuario), numeroCupos, usoCupoFanatico (boolean)
Métodos clave	getUsuario(), getNumeroCupos(), isUsoCupoFanatico()

5.2 Extensiones a Clases Existentes

Las siguientes clases del Proyecto 1 fueron extendidas para soportar el módulo de torneos:

Clase	Atributo(s) agregado(s)	Método(s) agregado(s)
Cliente	bonoDescuentoGanado, premioMonetarioPendiente	recibirBonoDescuento(), usarBonoDescuento(), aplicarBonoAVenta(), registrarPremioMonetario()
Empleado	bonoDescuentoGanado	recibirBonoDescuento(), usarBonoDescuento(), aplicarBonoAVenta()
Administrador	(ninguno nuevo)	crearTorneoAmistoso(), crearTorneoCompetitivo(), cancelarTorneo(), registrarEmpleado()
Cafe	torneos (List<Torneo>)	agregarTorneo(), eliminarTorneo(), getTorneos(), setTorneos(), consultarTorneosDisponibles(), validarCreacionTorneo(), buscarTorneoPorNombre()
SistemasDulcesDados	sugerencias (List<SugerenciaPlatillo>)	crearTorneoAmistoso(), crearTorneoCompetitivo(), inscribirEnTorneo(), desinscribirDeTorneo(), consultarTorneos(), consultarTorneosDisponibles(), aplicarBonoATorneo(), registrarCliente(), registrarEmpleadoPorAdministrador()

6. Enumeraciones del Sistema

Las siguientes enumeraciones están definidas en el paquete modelo y son utilizadas por varias clases del sistema:

Enumeración	Valores
DiaSemana	LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO
EstadoJuego	NUEVO, BUENO, FALTA_PIEZA, DESAPARECIDO
CategoriaJuego	CARTAS, TABLERO, ACCION
EstadoReserva	ACEPTADA, RECHAZADA, ACTIVA, CERRADA
TipoVenta	CAFETERIA, TIENDA_JUEGOS
TipoSolicitud	CAMBIO, INTERCAMBIO
EstadoSugerencia	PENDIENTE, APROBADA, RECHAZADA
Alergeno	MANI, GLUTEN, MARISCOS, LACTEOS, HUEVO
CategoriaPropuesta	BEBIDA, PASTELERIA

7. Relaciones entre Clases

7.1 Jerarquías de Herencia

Clase Padre	Clases Hijas
Usuario (abstracta)	Cliente, Empleado (abstracta), Administrador
Empleado (abstracta)	Mesero, Cocinero
ProductoVendible (abstracta)	JuegoVenta, ProductoMenu (abstracta)
ProductoMenu (abstracta)	Bebida, Pasteleria
Torneo (abstracta)	TorneoAmistoso, TorneoCompetitivo

7.2 Asociaciones Principales

Relación	Cardinalidad	Descripción
SistemasDulcesDados → Cafe	1 — 1	El sistema administra una única instancia del café.
SistemasDulcesDados → Usuario	1 — *	El sistema gestiona todos los usuarios registrados.
SistemasDulcesDados → PersistenciaSistema	1 — 1	El sistema usa una única instancia de persistencia.
Cafe → Mesa	1 — *	El café tiene múltiples mesas físicas.
Cafe → Empleado	1 — *	El café gestiona todos sus empleados.
Cafe → JuegoMesa	1 — *	El catálogo de préstamo del café.
Cafe → JuegoVenta	1 — *	El inventario de tienda del café.
Cafe → ProductoMenu	1 — *	El menú de cafetería.
Cafe → Torneo (*)	1 — *	El café contiene la lista de torneos activos. (*) Proyecto 2
Torneo → InscripcionTorneo (*)	1 — *	Cada torneo mantiene su lista de inscripciones. (*) Proyecto 2
Torneo → JuegoMesa (*)	1 — 1	Cada torneo se juega con un juego específico. (*) Proyecto 2
Mesa → ReservaMesa	1 — *	Historial de reservas por mesa.
Cliente → ReservaMesa	1 — *	Un cliente puede tener múltiples reservas.
JuegoMesa → CopiaJuegoPrestamo	1 — *	Cada juego tiene múltiples copias físicas.
Usuario → Prestamo	1 — *	Historial de préstamos por usuario.
Prestamo → DetallePrestamo	1 — 1..2	Cada préstamo cubre entre 1 y 2 juegos.
Usuario → Venta	1 — *	Historial de compras por usuario.

Relación	Cardinalidad	Descripción
Mesero → Venta	1 — *	Cada venta es registrada por un mesero.
Venta → ItemVenta	1 — *	Cada venta contiene uno o más ítems.
Empleado → Turno	1 — *	Historial de turnos por empleado.
Cliente * → JuegoMesa	* — *	Juegos favoritos de cada cliente (determina fanático).
Empleado * → JuegoMesa	* — *	Juegos favoritos de cada empleado.
Mesero * → JuegoMesa	* — *	Juegos que el mesero sabe explicar.
Pasteleria * → Alergeno	* — *	Alérgenos presentes en cada producto de pastelería.

8. Reglas de Negocio

8.1 Reglas del Proyecto 1 — Operación del Café

- Un cliente puede solicitar máximo dos juegos simultáneamente en préstamo.
- Es obligatorio tener una mesa activa para que un cliente pueda solicitar préstamos.
- No se permiten bebidas alcohólicas en mesas donde hay menores de edad.
- No se permiten juegos de acción en mesas donde hay bebidas calientes activas.
- Los juegos catalogados como difíciles requieren un mesero capacitado; de no haber uno disponible, el sistema genera una advertencia pero permite el préstamo.
- Los empleados tienen descuentos especiales en sus compras dentro del café.
- El administrador es el único que puede registrar empleados, aprobar sugerencias de platillos, asignar y modificar turnos, y gestionar el inventario.
- Las solicitudes de cambio de turno deben respetar el mínimo operativo: al menos 1 cocinero y 2 meseros de forma simultánea.

8.2 Reglas del Proyecto 2 — Torneos

- Solo el administrador puede crear torneos. El número de cupos no puede superar la capacidad real del juego, calculada como $\text{maxJugadores} \times \text{copiasDisponibles}$.
- Al crear un torneo, el sistema reserva automáticamente el 20% de los cupos (redondeado hacia arriba, usando `Math.ceil`) para jugadores fanáticos del juego.
- Un usuario es fanático de un torneo si tiene el juego del torneo en su lista de juegos favoritos.
- Los fanáticos usan primero el pool de cupos reservado. Cuando ese pool se agota, pasan al pool regular. Los usuarios no fanáticos siempre usan el pool regular.
- Un usuario puede inscribir entre 1 y 3 participantes en un mismo torneo, y no puede inscribirse dos veces en el mismo torneo.
- Al desinscribirse, todos los cupos tomados se liberan y se devuelven al pool correcto (fanático o regular) según el registro de la inscripción original.
- Los empleados pueden inscribirse a torneos siempre que no tengan turno asignado el día de realización del torneo.
- Los torneos competitivos son gratuitos para los empleados, pero los empleados no reciben el premio monetario aunque ganen.
- El premio monetario de un torneo competitivo se calcula sumando únicamente las tarifas de los cupos inscritos por clientes.
- El bono de descuento ganado en un torneo amistoso no es acumulable: `recibirBonoDescuento()` sobrescribe el valor existente.

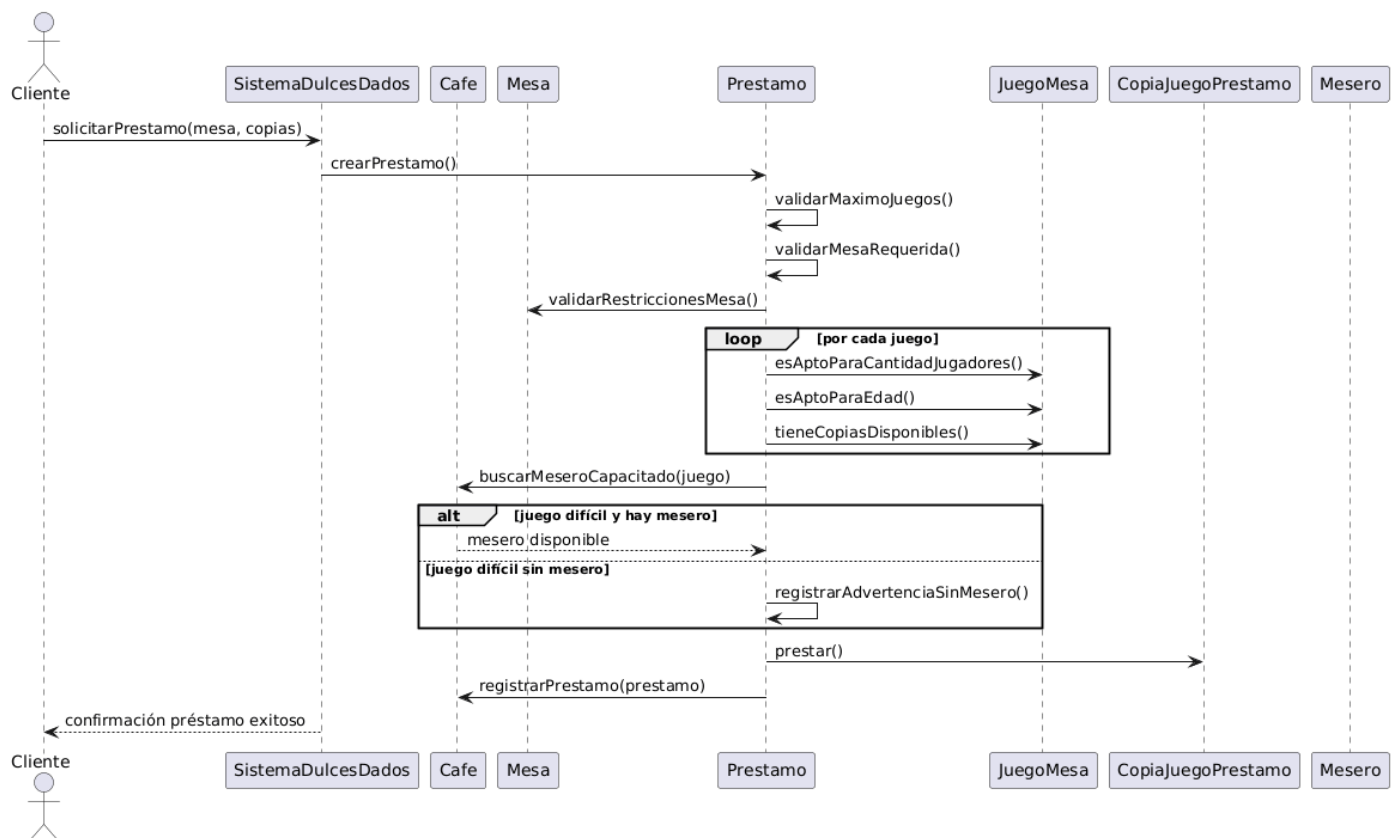
9. Diagramas de Secuencia

9.1 Solicitud de Préstamo de Juegos (Proyecto 1)

El flujo inicia cuando el cliente envía una solicitud al sistema a través del método solicitarPréstamo, proporcionando la mesa asociada y las copias de los juegos que desea. SistemaDulcesDados actúa como intermediario, delegando la responsabilidad al objeto Prestamo.

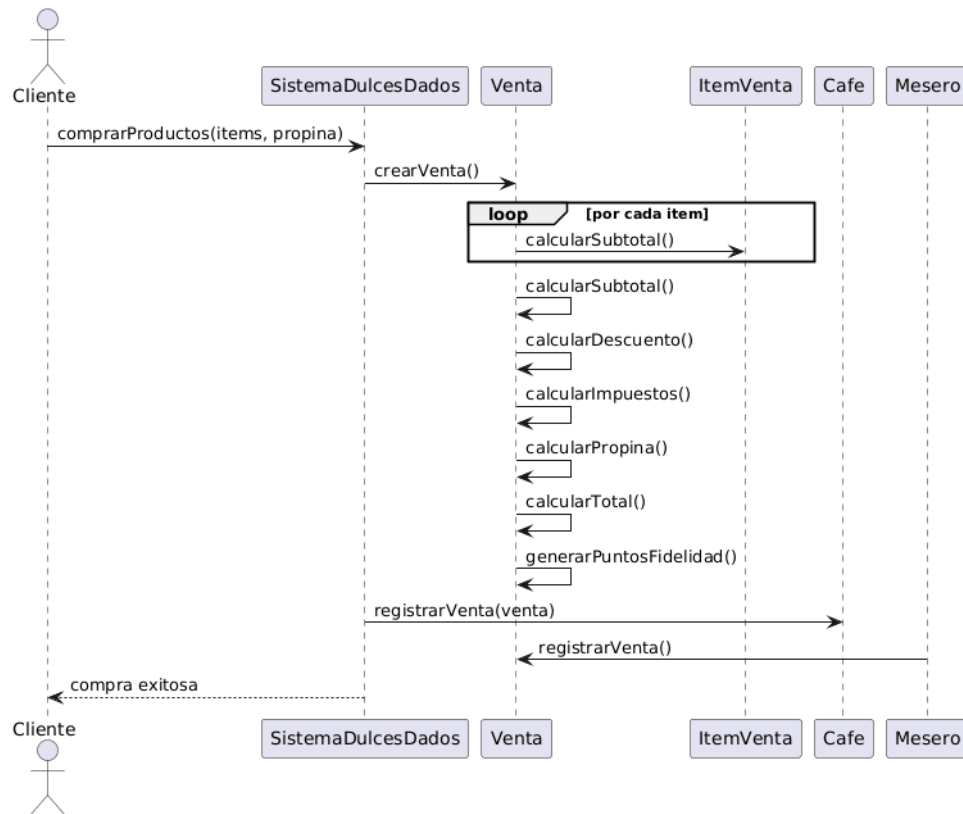
El objeto Prestamo centraliza todas las validaciones: que no se excedan dos juegos simultáneos, que el cliente tenga mesa asignada, que las restricciones de la mesa sean compatibles con los juegos, que cada JuegoMesa tenga el número de jugadores dentro del rango permitido, que sea apto para la edad de los participantes y que existan copias disponibles.

Para juegos difíciles, el Prestamo solicita al Cafe la búsqueda de un mesero capacitado. Si no hay uno disponible, se registra una advertencia sin bloquear el préstamo. Superadas todas las validaciones, las copias se marcan como no disponibles, el préstamo se registra en el Cafe y se confirma la operación al cliente.



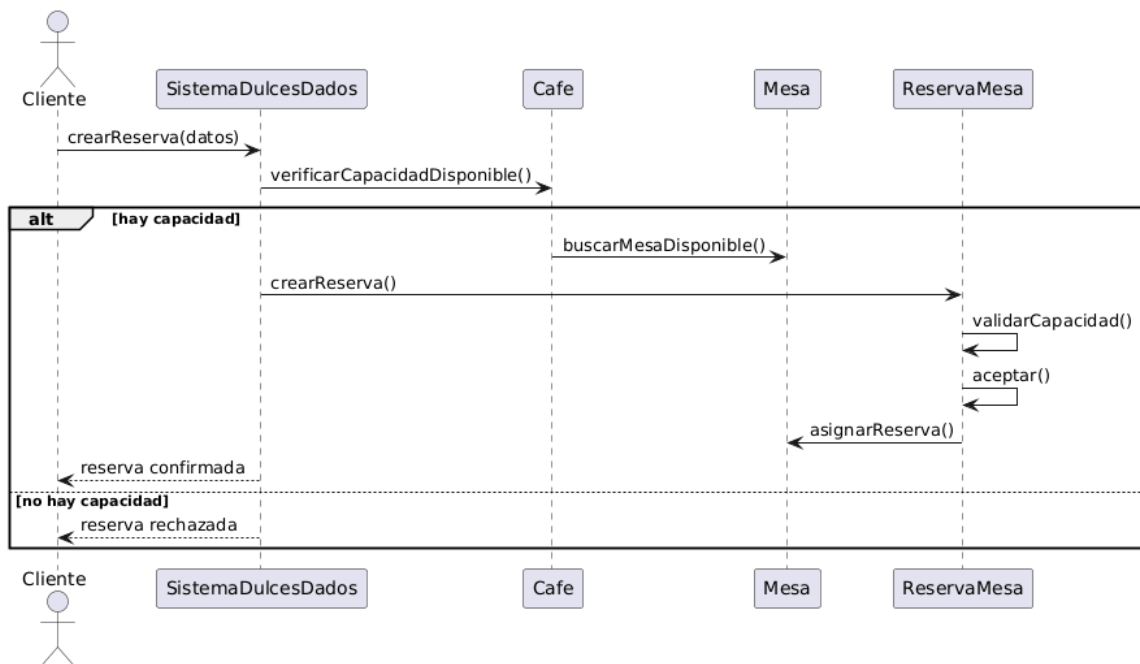
9.2 Compra de Productos (Proyecto 1)

El flujo inicia cuando el usuario solicita la compra con los ítems deseados y la propina. El sistema crea una instancia de Venta, recorre los ítems calculando subtotales individuales y luego calcula el subtotal general, descuentos, impuestos, propina y total. Se generan los puntos de fidelidad y la venta es registrada en el Cafe, formalizada por un Mesero como responsable de caja.



9.3 Reserva de Mesa (Proyecto 1)

El cliente envía la solicitud al sistema con el número de personas y características del grupo. El sistema consulta al Cafe si hay capacidad disponible; en caso afirmativo, busca una mesa adecuada y crea una instancia de ReservaMesa. Esta instancia valida la capacidad y las condiciones antes de aceptarse y asignarse a la mesa. Si no hay capacidad, la reserva se rechaza con notificación al usuario.



9.4 Creación de Torneo por el Administrador (Proyecto 2)

- El administrador selecciona la opción de crear torneo en MainAdministrador.
- Se recogen nombre, juego, cupos, día y valor económico mediante ValidadorEntradaConsola.
- MainAdministrador llama a sistema.crearTorneoAmistoso() o crearTorneoCompetitivo().
- SistemasDulcesDados verifica que sesionActual instanceof Administrador.
- Se delega a Administrador.crearTorneoXxx(..., cafe), que llama a Cafe.validarCreacionTorneo().
- Si el número de cupos es válido, se instancia el torneo, se calcula cuposFanaticos = ceil(cupos * 0.20) y se agrega a cafe.agregarTorneo().
- El torneo se retorna y el Main muestra confirmación al usuario.

9.5 Inscripción de Cliente a Torneo (Proyecto 2)

- El cliente selecciona la opción de inscripción en MainCliente.
- MainCliente llama a sistema.inscribirEnTorneo(torneo, numeroCupos).
- SistemasDulcesDados verifica que hay sesión activa y delega a torneo.inscribir(usuario, cupos).
- Torneo.puedeInscribirse() evalúa: rango 1-3 cupos, doble inscripción, restricción de turno para empleados y cupos totales disponibles.
- Torneo.esFanatico() consulta la lista juegosFavoritos del usuario.
- Se asignan cupos del pool fanático (si aplica) o del pool regular, y se crea el objeto InscripcionTorneo con el flag usoCupoFanatico.
- La inscripción se retorna al Main, que muestra confirmación.

9.6 Guardado Automático de Datos (Proyecto 2)

- Al salir de cualquier menú, el bloque finally del main llama a sistema.guardarDatos().
- guardarDatos() invoca persistencia.guardarUsuarios(usuarios) y persistencia.guardarCafe(cafe).
- guardarCafe desencadena métodos especializados: escribirCafeBasico, escribirJuegosMesa, escribirEmpleadosCafe, escribirTurnos, escribirReservas, escribirVentas, entre otros.
- Se llama también a persistencia.guardarTorneos(cafe.getTorneos()), que escribe cada torneo y sus inscripciones activas en torneos.txt.
- La caché de PersistenciaSistema se invalida para que la próxima carga refleje los datos actualizados.

10. Persistencia de Datos

El sistema utiliza archivos de texto plano ubicados en la carpeta src/datos como mecanismo de persistencia. Los archivos se crean automáticamente en la primera ejecución y se actualizan al finalizar cada sesión. El formato de cada archivo usa el carácter pipe (|) como delimitador entre campos, lo que facilita la lectura y depuración manual.

10.1 Archivos de Persistencia

Archivo	Proyecto	Contenido
usuarios.txt	P1	Tipo de usuario, documento, nombre, correo, login y contraseña de todos los usuarios.
cafe.txt	P1	Configuración básica del café (capacidad máxima).
juegos_mesa.txt	P1	Catálogo completo de juegos de mesa con sus atributos.
copias_prestamo.txt	P1	Copias físicas de cada juego y su estado actual.
juegos_venta.txt	P1	Inventario de juegos disponibles para venta en tienda.
menu.txt	P1	Productos del menú del café (bebidas y pastelería).
turnos.txt	P1	Turnos asignados a cada empleado.
reservas.txt	P1	Historial de reservas de mesa.
prestamos.txt	P1	Historial de préstamos y sus detalles por copia.
ventas.txt	P1	Historial de ventas e ítems de cada venta.
solicitudes_turno.txt	P1	Solicitudes de cambio o intercambio de turno.
sugerencias.txt	P1	Sugerencias de platillos de los empleados.
juegos_favoritos.txt	P1/P2	Juegos favoritos por usuario (necesario para lógica de fanáticos).
torneos.txt	P2	Torneos activos con tipo, nombre, día, juego, cupos, valor y sub-registros de inscripciones.
bonos.txt	P2	Bono de descuento activo por usuario.
premios.txt	P2	Premio monetario pendiente por usuario.

10.2 Mecanismo de Caché

PersistenciaSistema implementa un flag booleano cargado que se activa tras la primera lectura completa del sistema de archivos (método cargarTodo()). Las sucesivas llamadas a cargarUsuarios() o cargarCafe() durante la misma sesión retornan directamente desde los objetos en memoria, evitando accesos redundantes al disco. El caché se invalida con invalidarCache() cada vez que se realizan escrituras, garantizando consistencia si los datos se recargan dentro de la misma sesión.

10.3 Ciclo de Vida de los Datos

Cada vez que se inicia una aplicación (MainAdministrador, MainCliente o MainEmpleado), el sistema llama a inicializarSistema() → cargarDatos(). Este método carga los usuarios, el estado del café y los

torneos desde sus respectivos archivos. Al finalizar la sesión, el bloque finally garantiza la ejecución de guardarDatos(), que escribe todos los cambios de la sesión de forma atómica al terminar, sin persistencia concurrente por operación.

11. Interfaces de Consola

En el Proyecto 2 se implementaron tres clases Main independientes, una por cada rol de usuario. Todas siguen el mismo ciclo: inicialización del sistema, autenticación, bucle de menú y guardado al finalizar.

11.1 MainAdministrador

Requiere autenticación con rol de Administrador. Las opciones disponibles son: crear torneo amistoso, crear torneo competitivo, registrar cliente, registrar empleado (Mesero o Cocinero), listar torneos, listar usuarios y declarar ganador de un torneo.

11.2 MainCliente

Permite tanto el registro autónomo de nuevos clientes como el inicio de sesión de clientes existentes. Tras autenticarse, el cliente puede: consultar torneos disponibles, inscribirse a un torneo (indicando entre 1 y 3 cupos), desinscribirse de un torneo, y gestionar su lista de juegos favoritos.

11.3 MainEmpleado

Solo permite iniciar sesión (los empleados son registrados por el administrador). Tras autenticarse, el empleado puede consultar torneos disponibles y inscribirse a un torneo (validación automática de turno).

11.4 ValidadorEntradaConsola

Clase auxiliar con métodos estáticos que centralizan la validación de entradas desde Scanner, evitando duplicación entre los tres Main:

- leerEnteroEnRango(scanner, mensaje, min, max): repite la solicitud hasta obtener un entero válido dentro del rango.
- leerTextoNoVacio(scanner, mensaje): repite hasta que el texto no esté vacío ni en blanco.
- leerDoublePositivo(scanner, mensaje): repite hasta que el número sea mayor o igual a cero.

12. Pruebas Automatizadas

En el Proyecto 2 se implementaron pruebas automatizadas con JUnit 5 (Jupiter), organizadas en pruebas unitarias de la lógica de torneos y pruebas de integración basadas en las historias de usuario.

12.1 Pruebas Unitarias — TestTorneos

La clase TestTorneos (con 28 casos de prueba) verifica el comportamiento aislado del módulo de torneos, cubriendo las siguientes categorías:

Categoría	Casos cubiertos
Creación de torneos	Registro correcto en el café, cálculo de cuposFanaticos con ceil(20%), rechazo por cupos excesivos, rechazo sin copias disponibles.
Inscripción básica	Inscripción exitosa, actualización de cupos, rechazo de doble inscripción, rechazo de 0 cupos, rechazo de más de 3 cupos.
Fanáticos	Reconocimiento correcto de fanático, uso del pool especial, desbordamiento al pool regular cuando los cupos fanáticos se agotan.
Empleados en torneos	Rechazo de inscripción si tiene turno el día del torneo, inscripción exitosa si no tiene turno.
Desinscripción	Liberación correcta de cupos regulares, liberación de cupos fanáticos, fallo al desinscribirse sin inscripción previa.
Torneos competitivos	Tarifa cero para empleados, cálculo correcto del premio monetario excluyendo empleados.
Torneos amistosos	Descripción correcta del premio, otorgamiento del bono al ganador.
Capacidad	Verificación de torneo lleno, consulta de disponibilidad total de cupos.

12.2 Pruebas de Integración

Las pruebas de integración se encuentran en el paquete test.torneos y cubren tres áreas, cada una con su propio directorio de datos temporal que se elimina en @AfterEach para garantizar independencia entre pruebas:

- PruebasIntegracionUsuarios (12 pruebas): registro de clientes y empleados, unicidad del login, autenticación con credenciales válidas e inválidas, restricción de roles para operaciones protegidas.
- PruebasIntegracionTorneos (8 pruebas): creación de torneos a través del sistema con administrador autenticado, inscripción y desinscripción de clientes, reglas de fanáticos y turnos de empleados a nivel de sistema, consulta de torneos disponibles.
- PruebasIntegracionPersistencia (6 pruebas): ciclo completo guardar-cargar para usuarios, torneos amistosos, torneos competitivos e inscripciones; verificación de tipos e integridad de datos tras recarga en un sistema nuevo.

13. Decisiones de Diseño y Justificaciones

Decisión	Proyecto	Justificación
SistemasDulcesDatos como Fachada	P1/P2	Desacopla la capa de presentación de la lógica de dominio. Los Main no necesitan conocer la estructura interna del sistema; solo interactúan con métodos de alto nivel.
Herencia en Usuario y ProductoVendible	P1	Permite reutilizar atributos y comportamientos comunes, aplicando el principio DRY. Facilita el polimorfismo en operaciones que aplican a todos los tipos de usuario o producto.
Separación de inventarios de venta y préstamo	P1	Refleja la realidad del negocio: el stock de tienda y las copias de préstamo son entidades independientes con reglas propias.
Préstamo encapsula su propia validación	P1	Aplica el principio de responsabilidad única. Toda la lógica de validación de préstamos vive en Préstamo, facilitando su prueba y mantenimiento.
Torneo como clase abstracta con dos subtipos	P2	Aplica el principio Open/Closed: la lógica común de cupos e inscripciones está en la clase base; el comportamiento del premio se delega polimórficamente. Agregar un nuevo tipo de torneo solo requiere una nueva subclase.
Pool dual de cupos fanático/regular	P2	Refleja fielmente el requisito: garantiza el 20% reservado para fanáticos sin bloquear a los no fanáticos cuando hay cupos regulares disponibles.
InscripcionTorneo como clase separada	P2	Encapsula todos los datos de una inscripción (usuario, cupos, flag fanático) sin mezclar responsabilidades en Torneo. El flag usoCupoFanatico es esencial para devolver cupos al pool correcto al desinscribirse.
ValidadorEntradaConsola estático	P2	Centraliza la lógica de validación evitando duplicación entre los tres Main. Cualquier mejora se aplica automáticamente a todos los roles.
Persistencia en archivos .txt delimitados	P2	No requiere dependencias externas. Los archivos son legibles y editables directamente, facilitando la depuración. El formato pipe-delimitado es simple de parsear con split().
Caché lazy en PersistenciaSistema	P2	Evita lecturas redundantes del disco en operaciones que se ejecutan múltiples veces dentro de la misma sesión, mejorando el rendimiento sin complejidad adicional.

14. Conclusiones

El sistema desarrollado para el Board Game Café "Dulces & Dados" a lo largo de los Proyectos 1 y 2 constituye una solución de software modular, extensible y alineada con los principios de la programación orientada a objetos. La evolución del diseño entre ambas etapas evidencia que una arquitectura bien estructurada desde el inicio facilita significativamente la incorporación de nuevas funcionalidades sin comprometer el código existente.

En el Proyecto 1 se estableció una base sólida con la separación de responsabilidades en tres capas, la jerarquía de usuarios, la gestión de inventarios diferenciados y la encapsulación de la lógica de préstamos y ventas. Estas decisiones permitieron que en el Proyecto 2 el módulo de torneos se integrara con cambios acotados: nuevas clases en la jerarquía de Torneo, extensiones puntuales en Cliente, Empleado, Administrador y Café, y la incorporación de nuevos archivos en la capa de persistencia.

El patrón Fachada implementado en `SistemasDulcesDados` demostró su valor en el Proyecto 2: las tres interfaces de consola se construyeron de forma limpia y rápida, sin necesidad de conocer los detalles internos del dominio. La clase `ValidadorEntradaConsola` garantizó robustez en la entrada de datos sin duplicar código entre los tres flujos de usuario.

Las pruebas automatizadas con JUnit proporcionan una red de seguridad que facilita la detección temprana de errores ante futuras modificaciones del sistema.